NATIONAL UNIVERSITY
COLLEGE OF COMPUTING AND
INFORMATION TECHNOLOGIES

VER 1.0

COURSE MANUAL

INFORMATION ASSURANCE AND SECURITY

This course provides students with the theoretical knowledge and practical skills in the implementation of Information Assurance and Security providing relevant solutions for security hardening and information protection through computer vulnerability assessment, secure coding, and computer security hardening techniques.

BS – COMPUTER SCICENCE

BS – INFORMATION TECHNOLOGY

Security and System Administration Cluster

MODULE 6
OVERVIEW OF VULNERABILITY ASSESSMENT

Table of Contents

Lesson Topics

01

Principles of Vulnerability Assessment

A vulnerability assessment is a way you can discover, analyze and mitigate weakness within your attack surface to lessen the chance that attackers can exploit your network and gain unauthorized access to your systems and devices.

02

OWASP Top 10 Vulnerabilities

The OWASP Top 10 is a standard awareness document for developers and web application security. It represents a broad consensus about the most critical security risks to web applications.

LESSON 1

Principles of Vulnerability Assessment

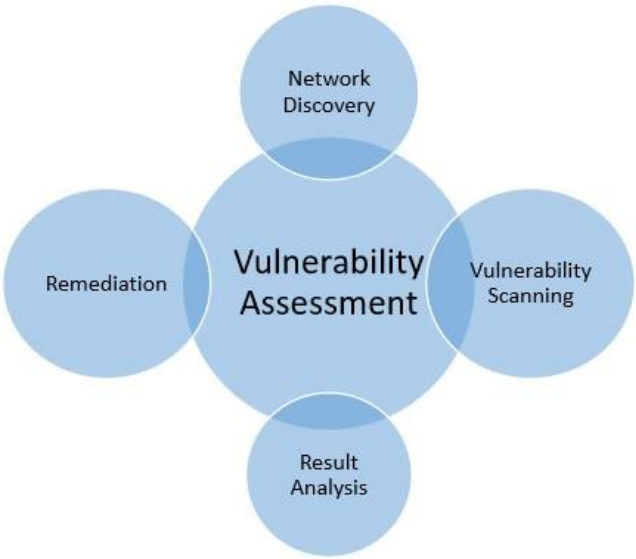
Learning Outcomes:

- Differentiate vulnerability assessment and penetration testing.
- Explain the purpose of vulnerability assessment.
- Describe the basic vulnerability assessment process.

Understanding Penetration Testing and Vulnerability Assessment

Penetration Testing Vs. Vulnerability Assessment

Generally, these two terms, i.e., Penetration Testing and Vulnerability assessment are used interchangeably by many people, either because of misunderstanding or marketing hype. But, both the terms are different from each other in terms of their objectives and other means.



Penetration Testing

Penetration testing replicates the actions of an external or/and internal cyber attacker/s that is intended to break the information security and hack the valuable data or disrupt the normal functioning of the organization. So, with the help of advanced tools and techniques, a penetration tester (also known as ethical hacker) makes an effort to control critical systems and acquire access to sensitive data.

Vulnerability Assessment

On the other hand, a vulnerability assessment is the technique of identifying (discovery) and measuring security vulnerabilities (scanning) in a given environment. It is a comprehensive assessment of the information security position (result analysis). Further, it identifies the potential weaknesses and provides the proper mitigation measures (remediation) to either remove those weaknesses or reduce below the risk level.

Penetration Testing	Vulnerability Assessments
Determines the scope of an attack.	Makes a directory of assets and resources in a given system.
Tests sensitive data collection.	Discovers the potential threats to each resource.
Gathers targeted information and/or inspect the system.	Allocates quantifiable value and significance to the available resources.
Cleans up the system and gives final report.	Attempts to mitigate or eliminate the potential vulnerabilities of valuable resources.
It is non-intrusive, documentation and environmental review and analysis.	Comprehensive analysis and through review of the target system and its environment.
It is ideal for physical environments and network architecture.	It is ideal for lab environments.
It is meant for critical real-time systems.	It is meant for non-critical systems.

Which Option is Ideal to Practice?

Both the methods have different functionality and approach, so it depends upon the security position of the respective system. However, because of the basic difference between penetration testing and vulnerability assessment, the second technique is more beneficial over the first one.

Vulnerability assessment identifies the weaknesses and gives solution to fix them. On the other hand, penetration testing only answers the question that "can anyone break-in the system security and if so, then what harm he can do?“

Further, a vulnerability assessment attempts to improve security system and develops a more mature, integrated security program. On the other hand, a penetration testing only gives a picture of your security program’s effectiveness.

As we have seen here, the vulnerability assessment is more beneficial and gives better result in comparison to penetration testing. But, experts suggest that, as a part of security management system, both techniques should be performed routinely to ensure a perfect secured environment.

OWASP Top 10 Vulnerabilities

Learning Outcomes:

- Describe the function of OWASP ZAP.
- Perform a basic vulnerability scan using OWASP ZAP.
- Interpret scan results ethically and responsibly.

OWASP Top 10 Web Application Risks

The OWASP Top 10 is a report, or "awareness document," that outlines security concerns around web application security. It is regularly updated to ensure it constantly features the 10 most critical risks facing organizations. OWASP recommends all companies to incorporate the document's findings into their corporate processes to ensure they minimize and mitigate the latest security risks.

The OWASP vulnerabilities report is formed on consensus from security experts all over the world. It ranks risks based on security defect frequency, vulnerability severity, and their potential impact. This provides developers and security professionals with insight into the most prominent risks and enables them to minimize the potential of the risks in their organizations' security practices.

Website: https://owasp.org/Top10/

A01:2025 Broken Access Control

Access control enforces policy such that users cannot act outside of their intended permissions. Failures typically lead to unauthorized information disclosure, modification or destruction of all data, or performing a business function outside the user's limits. Common access control vulnerabilities include:

- Violation of the principle of least privilege, commonly known as deny by default, where access should only be granted for particular capabilities, roles, or users, but is available to anyone.
- Bypassing access control checks by modifying the URL (parameter tampering or force browsing), internal application state, or the HTML page, or by using an attack tool that modifies API requests.
- Permitting viewing or editing someone else's account by providing its unique identifier (insecure direct object references)
- An accessible API with missing access controls for POST, PUT, and DELETE.
- Elevation of privilege. Acting as a user without being logged in or acting as an admin when logged in as a user.
- Metadata manipulation, such as replaying or tampering with a JSON Web Token (JWT) access control token, a cookie or hidden field manipulated to elevate privileges, or abusing JWT invalidation.
- CORS misconfiguration allows API access from unauthorized or untrusted origins.
- Force browsing (guessing URLs) to authenticated pages as an unauthenticated user or to privileged pages as a standard user.

How to prevent

Access control is only effective when implemented in trusted server-side code or serverless APIs, where the attacker cannot modify the access control check or metadata.

- Except for public resources, deny by default.
- Implement access control mechanisms once and reuse them throughout the application, including minimizing Cross-Origin Resource Sharing (CORS) usage.
- Model access controls should enforce record ownership rather than allowing users to create, read, update, or delete any record.
- Unique application business limit requirements should be enforced by domain models.
- Disable web server directory listing and ensure file metadata (e.g., .git) and backup files are not present within web roots.

- Log access control failures, alert admins when appropriate (e.g., repeated failures).
- Implement rate limits on API and controller access to minimize the harm from automated attack tooling.
- Stateful session identifiers should be invalidated on the server after logout. Stateless JWT tokens should be short-lived to minimize the window of opportunity for an attacker. For longer-lived JWTs, it's highly recommended to follow the OAuth standards to revoke access.
- Use well-established toolkits or patterns that provide simple, declarative access controls.

Developers and QA staff should include functional access control in their unit and integration tests.

A02:2025 Security Misconfiguration

Security misconfiguration is when a system, application, or cloud service is set up incorrectly from a security perspective, creating vulnerabilities.

The application might be vulnerable if:

- It is missing appropriate security hardening across any part of the application stack or improperly configured permissions on cloud services.
- Unnecessary features are enabled or installed (e.g., unnecessary ports, services, pages, accounts, testing frameworks, or privileges).
- Default accounts and their passwords are still enabled and unchanged.
- A lack of central configuration for intercepting excessive error messages. Error handling reveals stack traces or other overly informative error messages to users.
- For upgraded systems, the latest security features are disabled or not configured securely.
- Excessive prioritization of backward compatibility leading to insecure configuration.
- The security settings in the application servers, application frameworks (e.g., Struts, Spring, ASP.NET), libraries, databases, etc., are not set to secure values.
- The server does not send security headers or directives, or they are not set to secure values.

Without a concerted, repeatable application security configuration hardening process, systems are at a higher risk.

How to prevent

Secure installation processes should be implemented, including:

- A repeatable hardening process enabling the fast and easy deployment of another environment that is appropriately locked down. Development, QA, and production environments should all be configured identically, with different credentials used in each environment. This process should be automated to minimize the effort required to set up a new secure environment.
- A minimal platform without any unnecessary features, components, documentation, or samples. Remove or do not install unused features and frameworks.
- A task to review and update the configurations appropriate to all security notes, updates, and patches as part of the patch management process (see A03 Software Supply Chain Failures). Review cloud storage permissions (e.g., S3 bucket permissions).
- A segmented application architecture provides effective and secure separation between components or tenants, with segmentation, containerization, or cloud security groups (ACLs).
- Sending security directives to clients, e.g., Security Headers.
- An automated process to verify the effectiveness of the configurations and settings in all environments.
- Proactively add a central configuration to intercept excessive error messages as a backup.
- If these varifications are not automated, they should be manually verified annually at a minimum.

A03:2025 Software Supply Chain Failures

Software supply chain failures are breakdowns or other compromises in the process of building, distributing, or updating software. They are often caused by vulnerabilities or malicious changes in third-party code, tools, or other dependencies that the system relies on.

You are likely vulnerable if:

- If you do not carefully track the versions of all components that you use (both client-side and server-side). This includes components you directly use as well as nested (transitive) dependencies.
- If the software or one of its components is vulnerable, or unsupported/outdated. This includes the OS, web/application server, database management system (DBMS), applications, APIs and all components, runtime environments, and libraries.

- If you do not scan for vulnerabilities regularly and subscribe to security bulletins related to the components you use.
- If you do not have a change management process or tracking of changes within your supply chain, including tracking IDEs, IDE extensions and updates, changes to your organization's code repository, sandboxes, image and library repositories, the way artifacts are created and stored, etc. Every part of your supply chain should be documented, especially changes.
- If you have not hardened every part of your supply chain, with a special focus on access control and the application of least privilege.
- If your supply chain systems do not have any separation of duty. No single person should be able to write code and promote it all the way to production without oversight from another human being.
- If developers, DevOps, or infrastructure professionals are allowed to download and use components from untrusted sources, for use in production.
- If you do not fix or upgrade the underlying platform, frameworks, and dependencies in a risk-based, timely fashion. This commonly happens in environments when patching is a monthly or quarterly task under change control, leaving organizations open to days or months of unnecessary exposure before fixing vulnerabilities.
- If software developers do not test the compatibility of updated, upgraded, or patched libraries.
- If you do not secure the configurations of every part of your system (see A02:2025-Security Misconfiguration).
- If you have a complex CI/CD pipeline that uses many components but has weaker security than the rest of your application.

How to prevent

There should be a patch management process in place to:

- Know your Software Bill of Materials (SBOM) of your entire software and manage the SBOM-dictionary centrally.
- Track not just your own dependencies, but their (transitive) dependencies, and so on.
- Remove unused dependencies, unnecessary features, components, files, and documentation. Attack surface reduction.

- Continuously inventory the versions of both client-side and server-side components (e.g., frameworks, libraries) and their dependencies using tools like versions, OWASP Dependency Check, retire.js, etc.
- Continuously monitor sources like Common Vulnerability and Exposures (CVE) and National Vulnerability Database (NVD) for vulnerabilities in the components you use. Use software composition analysis, software supply chain, or security-focused SBOM tools to automate the process. Subscribe to email alerts for security vulnerabilities related to components you use.
- Only obtain components from official (trusted) sources over secure links. Prefer signed packages to reduce the chance of including a modified, malicious component (see A08:2025-Software and Data Integrity Failures).
- Deliberately choosing which version of a dependency you use and upgrading only when there is need.
- Monitor for libraries and components that are unmaintained or do not create security patches for older versions. If patching is not possible, consider deploying a virtual patch to monitor, detect, or protect against the discovered issue.
- Update your CI/CD, IDE, and any other developer tooling regularly
- Treat components in your CI/CD pipeline as part of this process; harden them, monitor them, and document changes accordingly

There should be a change management process or tracking system in place to track changes to:

- Your CI/CD settings (all build tools and pipeline)
- Your code repository
- Sandbox areas
- Developer IDEs
- Your SBOM tooling, and created artifacts
- Your logging systems and logs
- Third party integrations, such as SaaS
- Artifact repository
- Container registry

Harden the following systems, which includes enabling MFA and locking down IAM:

- Your code repository (which includes not checking in secrets, protecting branches, backups)
- Developer workstations (regular patching, MFA, monitoring, and more)

- Your build server & CI/CD (separation of duties, access control, signed builds, environment-scoped secrets, tamper-evident logs, more)
- Your artifacts (ensure integrity via providence, signing, and time stamping, promote artifacts rather than rebuilding for each environment, ensure builds are immutable)
- Infrastructure as code is managed like all code, including use of PRs and version control

Every organization must ensure an ongoing plan for monitoring, triaging, and applying updates or configuration changes for the lifetime of the application or portfolio.

A04:2025 Cryptographic Failures

Generally speaking, all data in transit should be encrypted at the transport layer (OSI layer 4). Previous hurdles such as CPU performance and private key/certificate management are now handled by CPUs having instructions designed to accelerate encryption (eg: AES support) and private key and certificate management being simplified by services like LetsEncrypt.org with major cloud vendors providing even more tightly integrated certificate management services for their specific platforms.

Beyond securing the transport layer, it is important to determine what data needs encryption at rest as well as what data needs extra encryption in transit (at the application layer, OSI layer 7). For example, passwords, credit card numbers, health records, personal information, and business secrets require extra protection, especially if that data falls under privacy laws, e.g., EU's General Data Protection Regulation (GDPR), or regulations such as PCI Data Security Standard (PCI DSS). For all such data:

- Are any old or weak cryptographic algorithms or protocols used either by default or in older code?
- Are default crypto keys in use, are weak crypto keys generated, are keys re-used, or is proper key management and rotation missing?
- Are crypto keys checked into source code repositories?
- Is encryption not enforced, e.g., are any HTTP headers (browser) security directives or headers missing?
- Is the received server certificate and the trust chain properly validated?

- Are initialization vectors ignored, reused, or not generated sufficiently secure for the cryptographic mode of operation? Is an insecure mode of operation such as ECB in use? Is encryption used when authenticated encryption is more appropriate?
- Are passwords being used as cryptographic keys in the absence of a password based key derivation function?
- Is randomness used for cryptographic purposes that was not designed to meet cryptographic requirements? Even if the correct function is chosen, does it need to be seeded by the developer, and if not, has the developer over-written the strong seeding functionality built into it with a seed that lacks sufficient entropy/unpredictability?
- Are deprecated hash functions such as MD5 or SHA1 in use, or are non-cryptographic hash functions used when cryptographic hash functions are needed?
- Are deprecated cryptographic padding methods such as PKCS number 1 v1.5 in use?
- Are cryptographic error messages or side channel information exploitable, for example in the form of padding oracle attacks?
- Can the cryptographic algorithm be downgraded or bypassed?

See references ASVS: Cryptography (V11), Secure Communication (V12) and Data Protection (V14).

How to prevent

Do the following, at a minimum, and consult the references:

- Classify and label data processed, stored, or transmitted by an application. Identify which data is sensitive according to privacy laws, regulatory requirements, or business needs.
- Store your most sensitive keys in a hardware or cloud-based HSM.
- Use well-trusted implementations of cryptographic algorithms whenever possible.
- Don't store sensitive data unnecessarily. Discard it as soon as possible or use PCI DSS compliant tokenization or even truncation. Data that is not retained cannot be stolen.
- Make sure to encrypt all sensitive data at rest.
- Ensure up-to-date and strong standard algorithms, protocols, and keys are in place; use proper key management.
- Encrypt all data in transit with secure protocols such as TLS with forward secrecy (FS) ciphers, cipher prioritization by the server, and secure parameters. Enforce encryption using directives like HTTP Strict Transport Security (HSTS).

- Disable caching for responses that contain sensitive data. This includes caching in your CDN, web server, and any application caching (eg: Redis).
- Apply required security controls as per the data classification.
- Do not use unencrypted protocols such as FTP and SMTP.
- Store passwords using strong adaptive and salted hashing functions with a work factor (delay factor), such as Argon2, scrypt, bcrypt (legacy systems) or PBKDF2-HMAC-SHA-256. For legacy systems using bcrypt.
- Initialization vectors must be chosen appropriate for the mode of operation. This could mean using a CSPRNG (cryptographically secure pseudo random number generator). For modes that require a nonce, the initialization vector (IV) does not need a CSPRNG. In all cases, the IV should never be used twice for a fixed key.
- Always use authenticated encryption instead of just encryption.
- Keys should be generated cryptographically randomly and stored in memory as byte arrays. If a password is used, then it must be converted to a key via an appropriate password base key derivation function.
- Ensure that cryptographic randomness is used where appropriate and that it has not been seeded in a predictable way or with low entropy. Most modern APIs do not require the developer to seed the CSPRNG to be secure.
- Avoid deprecated cryptographic functions, block building methods and padding schemes, such as MD5, SHA1, Cipher Block Chaining Mode (CBC), PKCS number 1 v1.5.
- Ensure settings and configurations meet security requirements by having them reviewed by security specialists, tools designed for this purpose, or both.
- Consider to prepare for post quantum cryptography (PQC), see references (ENISA, NIST)

A05:2025 - Injection

An injection vulnerability is a system flaw that allows an attacker to insert malicious code or commands (such as SQL or shell code) into a program's input fields, tricking the system into executing the code or commands as if it were part of the system. This can lead to truly dire consequences.

An application is vulnerable to attack when:

- User-supplied data is not validated, filtered, or sanitized by the application.

- Dynamic queries or non-parameterized calls without context-aware escaping are used directly in the interpreter.
- Unsanitized data is used within object-relational mapping (ORM) search parameters to extract additional, sensitive records.
- Potentially hostile data is directly used or concatenated. The SQL or command contains the structure and malicious data in dynamic queries, commands, or stored procedures.

Some of the more common injections are SQL, NoSQL, OS command, Object Relational Mapping (ORM), LDAP, and Expression Language (EL) or Object Graph Navigation Library (OGNL) injection. The concept is identical among all interpreters. Detection is best achieved by a combinatinon of source code review along with automated testing (including fuzzing) of all parameters, headers, URL, cookies, JSON, SOAP, and XML data inputs. The addition of static (SAST), dynamic (DAST), and interactive (IAST) application security testing tools into the CI/CD pipeline can also be helpful to identify injection flaws before production deployment.

How to prevent

The best means to prevent injection requires keeping data separate from commands and queries:

- The preferred option is to use a safe API, which avoids using the interpreter entirely, provides a parameterized interface, or migrates to Object Relational Mapping Tools (ORMs). Note: Even when parameterized, stored procedures can still introduce SQL injection if PL/SQL or T-SQL concatenates queries and data or executes hostile data with EXECUTE IMMEDIATE or exec().

When it is not possible to separate the data from commands, you can reduce threats using the following techniques.

- Use positive server-side input validation. This is not a complete defense as many applications require special characters, such as text areas or APIs for mobile applications.
- For any residual dynamic queries, escape special characters using the specific escape syntax for that interpreter. Note: SQL structures such as table names, column names, and so on cannot be escaped, and thus user-supplied structure names are dangerous. This is a common issue in report-writing software.

A06:2025 - Insecure Design

Insecure design is a broad category representing different weaknesses, expressed as “missing or ineffective control design.” Insecure design is not the source for all other Top Ten risk categories. Note that there is a difference between insecure design and insecure implementation. We differentiate between design flaws and implementation defects for a reason, they have different root causes, take place at different times in the development process, and have different remediations. A secure design can still have implementation defects leading to vulnerabilities that may be exploited. An insecure design cannot be fixed by a perfect implementation as needed security controls were never created to defend against specific attacks. One of the factors that contributes to insecure design is the lack of business risk profiling inherent in the software or system being developed, and thus the failure to determine what level of security design is required.

Three key parts of having a secure design are:
1. Gathering Requirements and Resource Management
2. Creating a Secure Design
3. Having a Secure Development Lifecycle

Requirements and Resource Management
Collect and negotiate the business requirements for an application with the business, including the protection requirements concerning confidentiality, integrity, availability, and authenticity of all data assets and the expected business logic. Take into account how exposed your application will be and if you need segregation of tenants (beyond those needed for access control). Compile the technical requirements, including functional and non-functional security requirements. Plan and negotiate the budget covering all design, build, testing, and operation, including security activities.

Secure Design
Secure design is a culture and methodology that constantly evaluates threats and ensures that code is robustly designed and tested to prevent known attack methods. Threat modeling should be integrated into refinement sessions (or similar activities); look for changes in data flows and access control or other security controls. In the user story development, determine the correct flow and failure states, ensure they are well understood and agreed upon by the responsible and impacted parties.

Analyze assumptions and conditions for expected and failure flows to ensure they remain accurate and desirable. Determine how to validate the assumptions and enforce conditions needed for proper behaviors. Ensure the results are documented in the user story. Learn from mistakes and offer positive incentives to promote improvements. Secure design is neither an add-on nor a tool that you can add to software.

Secure Development Lifecycle

Secure software requires a secure development lifecycle, a secure design pattern, a paved road methodology, a secure component library, appropriate tooling, threat modeling, and incident post-mortems that are used to improve the process. Reach out to your security specialists at the beginning of a software project, throughout the project, and for ongoing software maintenance. Consider leveraging the OWASP Software Assurance Maturity Model (SAMM) to help structure your secure software development efforts.

How to prevent

- Establish and use a secure development lifecycle with AppSec professionals to help evaluate and design security and privacy-related controls
- Establish and use a library of secure design patterns or paved-road components
- Use threat modeling for critical parts of the application such as authentication, access control, business logic, and key flows
- Integrate security language and controls into user stories
- Integrate plausibility checks at each tier of your application (from frontend to backend)
- Write unit and integration tests to validate that all critical flows are resistant to the threat model. Compile use-cases and misuse-cases for each tier of your application.
- Segregate tier layers on the system and network layers, depending on the exposure and protection needs
- Segregate tenants robustly by design throughout all tiers

A07:2025 - Authentication Failures

When an attacker is able to trick a system into recognizing an invalid or incorrect user as legitimate, this vulnerability is present. There may be authentication weaknesses if the application it:

- Permits brute force or other automated, scripted attacks that are not quickly blocked.

- Permits automated attacks such as credential stuffing, where the attacker has a breached list of valid usernames and passwords. More recently this type of attack has been expanded to include hybrid password attacks credential stuffing (also known as password spray attacks), where the attacker uses variations or increments of spilled credentials to gain access, for instance trying Password1!, Password2!, Password3! and so on.
- Permits default, weak, or well-known passwords, such as "Password1" or "admin" username with an "admin" password.
- Allows users to create new accounts with already known-breached credentials.
- Allows use of weak or ineffective credential recovery and forgot-password processes, such as "knowledge-based answers," which cannot be made safe.
- Uses plain text, encrypted, or weakly hashed passwords data stores (see A04:2025-Cryptographic Failures).
- Has missing or ineffective multi-factor authentication.
- Allows use of weak or ineffective fallbacks if multi-factor authentication is not available.
- Exposes session identifier in the URL, a hidden field, or another insecure location that is accessible to the client.
- Reuses the same session identifier after successful login.
- Does not correctly invalidate user sessions or authentication tokens (mainly single sign-on (SSO) tokens) during logout or a period of inactivity.

How to prevent

- Where possible, implement and enforce use of multi-factor authentication to prevent automated credential stuffing, brute force, and stolen credential reuse attacks.
- Where possible, encourage and enable the use of password managers, to help users make better choices.
- Do not ship or deploy with any default credentials, particularly for admin users.
- Implement weak password checks, such as testing new or changed passwords against the top 10,000 worst passwords list.
- During new account creation and password changes validate against lists of known breached credentials (eg: using haveibeenpwned.com).
- Align password length, complexity, and rotation policies with National Institute of Standards and Technology (NIST) 800-63b's guidelines in section 5.1.1 for Memorized Secrets or other modern, evidence-based password policies.

- Do not force human beings to rotate passwords unless you suspect breach. If you suspect breach, force password resets immediately.
- Ensure registration, credential recovery, and API pathways are hardened against account enumeration attacks by using the same messages for all outcomes ("Invalid username or password.").
- Limit or increasingly delay failed login attempts but be careful not to create a denial of service scenario. Log all failures and alert administrators when credential stuffing, brute force, or other attacks are detected or suspected.
- Use a server-side, secure, built-in session manager that generates a new random session ID with high entropy after login. Session identifiers should not be in the URL, be securely stored in a secure cookie, and invalidated after logout, idle, and absolute timeouts.
- Ideally, use a premade, well-trusted system to handle authentication, identity, and session management. Transfer this risk whenever possible by buying and utilizing a hardened and well tested system.

A08:2025 - Software or Data Integrity Failures

Software and data integrity failures relate to code and infrastructure that does not protect against invalid or untrusted code or data being treated as trusted and valid. An example of this is where an application relies upon plugins, libraries, or modules from untrusted sources, repositories, and content delivery networks (CDNs). An insecure CI/CD pipeline without consuming and providing software integety checks can introduce the potential for unauthorized access, insecure or malicious code, or system compromise. OneAnother example for this is a CI/CD that pulls code or artifacts from untrusted places and/or doesn't verify them before use (by checking the signature or similar mechanism). Lastly, many applications now include auto-update functionality, where updates are downloaded without sufficient integrity verification and applied to the previously trusted application. Attackers could potentially upload their own updates to be distributed and run on all installations. Another example is where objects or data are encoded or serialized into a structure that an attacker can see and modify is vulnerable to insecure deserialization.

How to prevent

- Use digital signatures or similar mechanisms to verify the software or data is from the expected source and has not been altered.
- Ensure libraries and dependencies, such as npm or Maven, are only consuming trusted repositories. If you have a higher risk profile, consider hosting an internal known-good repository that's vetted.
- Ensure that there is a review process for code and configuration changes to minimize the chance that malicious code or configuration could be introduced into your software pipeline.
- Ensure that your CI/CD pipeline has proper segregation, configuration, and access control to ensure the integrity of the code flowing through the build and deploy processes.
- Ensure that unsigned or unencrypted serialized data is not received from untrusted clients and subsequently used without some form of integrity check or digital signature to detect tampering or replay of the serialized data.

A09:2025 - Security Logging & Alerting Failures

Without logging and monitoring, attacks and breaches cannot be detected, and without alerting it is very difficult to respond quickly and effectively during a security incident. Insufficient logging, continuous monitoring, detection, and alerting to initiate active responses occurs any time:

- Auditable events, such as logins, failed logins, and high-value transactions, are not logged or logged inconsistently (for instance, only logging successful logins, but not failed attempts).
- Warnings and errors generate no, inadequate, or unclear log messages.
- The integrity of logs is not properly protected from tampering.
- Logs of applications and APIs are not monitored for suspicious activity.
- Logs are only stored locally, and not properly backedup.
- Appropriate alerting thresholds and response escalation processes are not in place or effective. Alerts are not received or reviewed within a reasonable amount of time.
- Penetration testing and scans by dynamic application security testing (DAST) tools (such as Burp or ZAP) do not trigger alerts.
- The application cannot detect, escalate, or alert for active attacks in real-time or near real-time.

- You are vulnerable to sensitive information leakage by making logging and alerting events visible to a user or an attacker (see A01:2025-Broken Access Control), or by logging sensitive information that should not be logged (such as PII or PHI).
- You are vulnerable to injections or attacks on the logging or monitoring systems if log data is not correctly encoded.
- The application is missing or mishandling errors and other exceptional conditions, such that the system is unaware there was an error, and is therefore unable to log there was a problem.
- Adequate 'use cases' for issuing alerts are missing or outdated to recognize a special situation.
- Too many false positive alerts make it impossible to distinguish important alerts from unimportant ones, resulting in them being recognized too late or not at all (physical overload of the SOC team).
- Detected alerts cannot be processed correctly because the playbook for the use case is incomplete, out of date, or missing.

How to prevent

Developers should implement some or all the following controls, depending on the risk of the application:

- Ensure all login, access control, and server-side input validation failures can be logged with sufficient user context to identify suspicious or malicious accounts and held for enough time to allow delayed forensic analysis.
- Ensure that every part of your app that contains a security control is logged, whether it succeeds or fails.
- Ensure that logs are generated in a format that log management solutions can easily consume.
- Ensure log data is encoded correctly to prevent injections or attacks on the logging or monitoring systems.
- Ensure all transactions have an audit trail with integrity controls to prevent tampering or deletion, such as append-only database tables or similar.
- Ensure all transactions that throw an error are rolled back and started over. Always fail closed.
- If your application or its users behave suspiciously, issue an alert. Create guidance for your developers on this topic so they can code against this or buy a system for this.
- DevSecOps and security teams should establish effective monitoring and alerting use cases including playbooks such that suspicious activities are detected and responded to quickly by the Security Operations Center (SOC) team.

- Add ‘honeytokens’ as traps for attackers into your application e.g. into the database, data, as real and/or technical user identity. As they are not used in normal business, any access generates logging data that can be alerted with nearly no false positives.
- Behavior analysis and AI support could be optionally an additional technique to support low rates of false positives for alerts.
- Establish or adopt an incident response and recovery plan, such as National Institute of Standards and Technology (NIST) 800-61r2 or later. Teach your software developers what application attacks and incidents look like, so they can report them.

There are commercial and open-source application protection products such as the OWASP ModSecurity Core Rule Set, and open-source log correlation software, such as the Elasticsearch, Logstash, Kibana (ELK) stack, that feature custom dashboards and alerting that may help you combat these issues. There are also commercial observability tools that can help you respond to or block attacks in close to real-time.

A10:2025 - Mishandling of Exceptional Conditions

Mishandling exceptional conditions in software happens when programs fail to prevent, detect, and respond to unusual and unpredictable situations, which leads to crashes, unexpected behavior, and sometimes vulnerabilities. This can involve one or more of the following 3 failings; the application doesn’t prevent an unusual situation from happening, it doesn’t identify the situation as it is happening, and/or it responds poorly or not at all to the situation afterwards.

Exceptional conditions can be caused by missing, poor, or incomplete input validation, or late, high level error handling instead at the functions where they occur, or unexpected environmental states such as memory, privilege, or network issues, inconsistent exception handling, or exceptions that are not handled at all, allowing the system to fall into an unknown and unpredictable state. Any time an application is unsure of its next instruction, an exceptional condition has been mishandled. Hard-to-find errors and exceptions can threaten the security of the whole application for a long time.

Many different security vulnerabilities can happen when we mishandle exceptional conditions, such as logic bugs, overflows, race conditions, fraudulent transactions, or issues with memory, state, resource, timing, authentication, and authorization. These types of vulnerabilities can negatively affect the confidentiality, availability, and/or integrity of a system or it's data. Attackers manipulate an application's flawed error handling to strike this vulnerability.

How to prevent

In order to handle an exceptional condition properly we must plan for such situations (expect the worst). We must 'catch' every possible system error directly at the place where they occur and then handle it (which means do something meaningful to solve the problem and ensure we recover from the issue). As part of the handling, we should include throwing an error (to inform the user in an understandable way), logging of the event, as well as issuing an alert if we feel that is justified. We should also have a global exception handler in place in case there is ever something we have missed. Ideally, we would also have monitoring and/or observability tooling or functionality that watches for repeated errors or patterns that indicate an on-going attack, that could issue a response, defense, or blocking of some kind. This can help us block and respond to scripts and bots that focus on our error handling weaknesses.

Catching and handling exceptional conditions ensures that the underlying infrastructure of our programs are not left to deal with unpredictable situations. If you are part way through a transaction of any kind, it is extremely important that you roll back every part of the transaction and start again (also known as failing closed). Attempting to recover a transaction part way through is often where we create unrecoverable mistakes.

Whenever possible, add rate limiting, resource quotas, throttling, and other limits wherever possible, to prevent exceptional conditions in the first place. Nothing in information technology should be limitless, as this leads to a lack of application resilience, denial of service, successful brute force attacks, and extraordinary cloud bills. \ Consider whether identical repeated errors, above a certain rate, should only be outputted as statistics showing how often they have occurred and in what time frame. This information should be appended to the original message so as not to interfere with automated logging and monitoring, see A09:2025 Seurity Logging & Alerting Failures.

On top of this, we would want to include strict input validation (with sanitization or escaping for potentially hazardous characters that we must accept), and centralized error handling, logging, monitoring, and alerting, and a global exception handler. One application should not multiple functions for handling exceptional conditions, it should be performed in one place, the same way each time. We should also create project security requirements for all the advice in this section, perform threat modelling and/or secure design review activities in the design phase of our projects, perform code review or static analysis, as well as execute stress, performance, and penetration testing of the final system.

If possible, your entire organization should handle exceptional conditions in the same way, as it makes it easier to review and audit code for errors in this important security control.

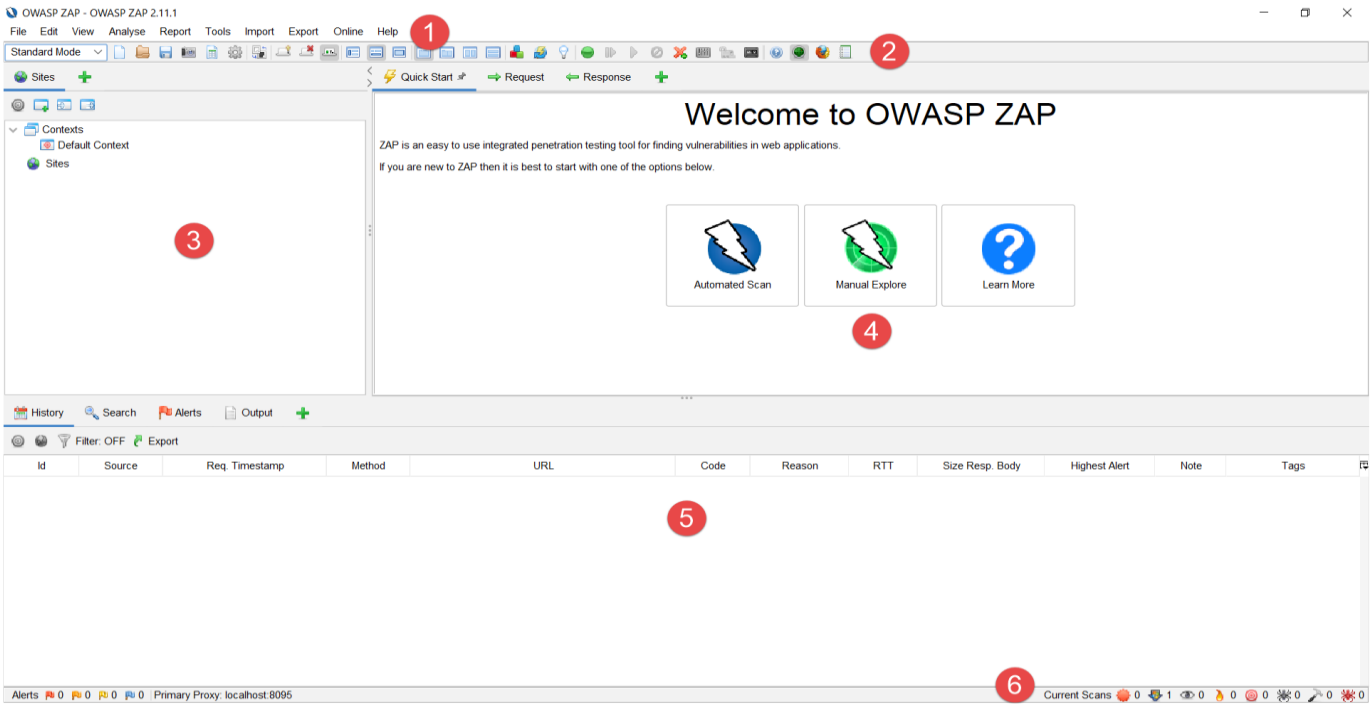
OWASP ZAP

The OWASP Zed Attack Proxy (ZAP) is one of the world's most popular web application security testing tools. It is made available for free as an open-source project and is contributed to and maintained by OWASP. The Open Web Application Security Project (OWASP) is a vendor-neutral, non-profit group of volunteers dedicated to making web applications more secure. The OWASP ZAP tool can be used during web application development by web developers or by experienced security experts during penetration tests to assess web applications for vulnerabilities.

The OWASP Zed Attack Proxy is a Java-based tool that comes with an intuitive graphical interface, allowing web application security testers to perform fuzzing, scripting, spidering, and proxying in order to attack web apps. Being a Java tool means that it can be made to run on most operating systems that support Java. ZAP can be found by default within the Kali Linux Penetration Testing Operating System, or it can be download from here and run-on OSs that have Java installed. The OWASP ZAP proxy borrows heavily in GUI appearance from the Paros Proxy Lightweight Web Application security testing tool.

OWASP ZAP Desktop User Interface

The ZAP Desktop UI is composed of the following elements:



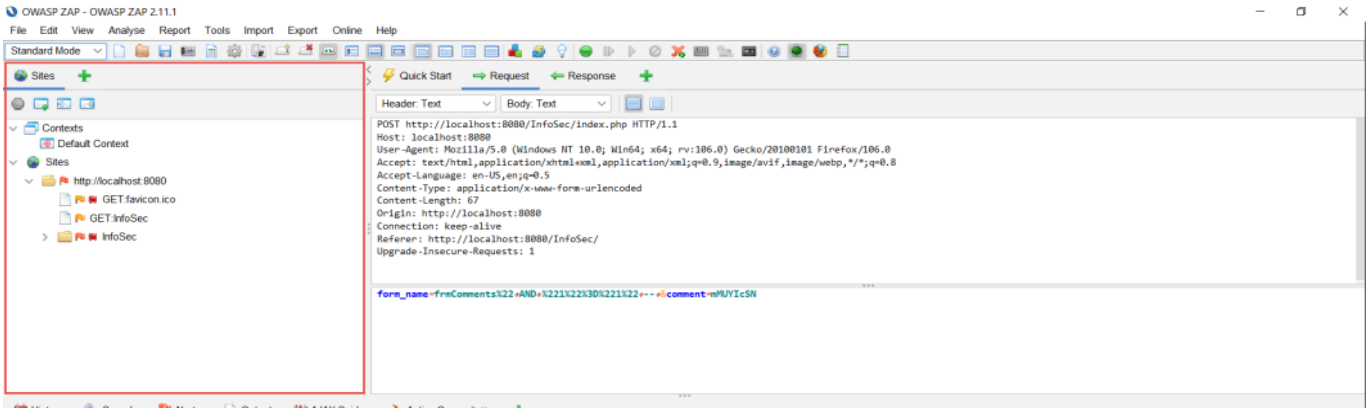
1. Menu Bar – Provides access to many of the automated and manual tools.
2. Toolbar – Includes buttons which provide easy access to most commonly used features.
3. Tree Window – Displays the Sites tree and the Scripts tree.
4. Workspace Window – Displays requests, responses, and scripts and allows you to edit them.
5. Information Window – Displays details of the automated and manual tools.
6. Footer – Displays a summary of the alerts found and the status of the main automated tools.

IMPORTANT: You should only use ZAP to attack an application you have permission to test with an active attack. Because this is a simulation that acts like a real attack, actual damage can be done to a site's functionality, data, etc. If you are worried about using ZAP, you can prevent it from causing harm (though ZAP's functionality will be significantly reduced) by switching to safe mode.

OWASP ZAP Tabs

Sites Tab

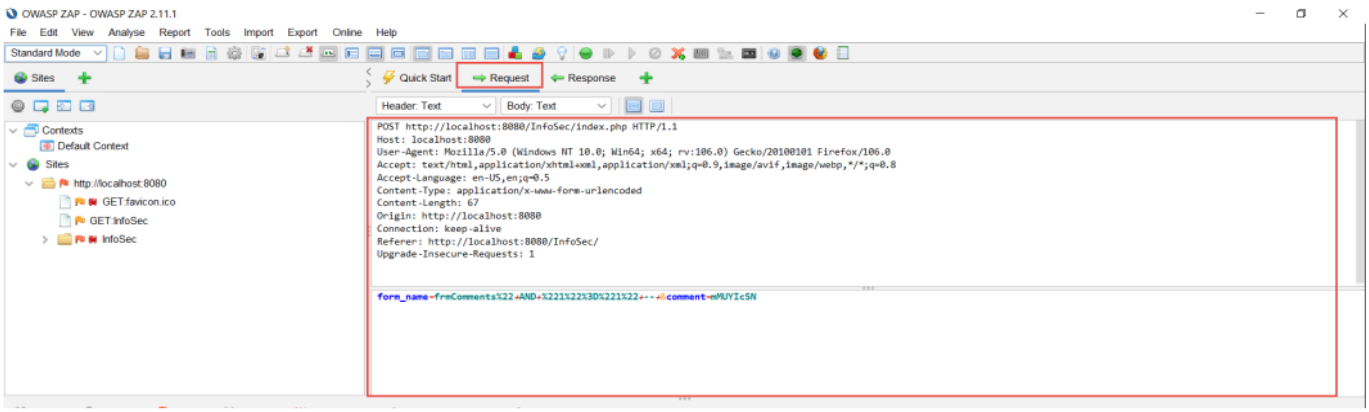
The sites tab contains two trees, one for Contexts and the other for the Sites Tree.



- Contexts are a way of relating a set of URLs together. You can define any contexts you like, but it is expected that a context will correspond to a web application.
- The Sites Tree is ZAP's internal representation of the sites that you access and is displayed in the Sites tab. If it does not accurately reflect the sites, then ZAP will not be able to attack them effectively. Each node in the tree represents a different piece of functionality in a site. By default, ZAP will create unique nodes in the tree based on the HTTP method and the parameter names.

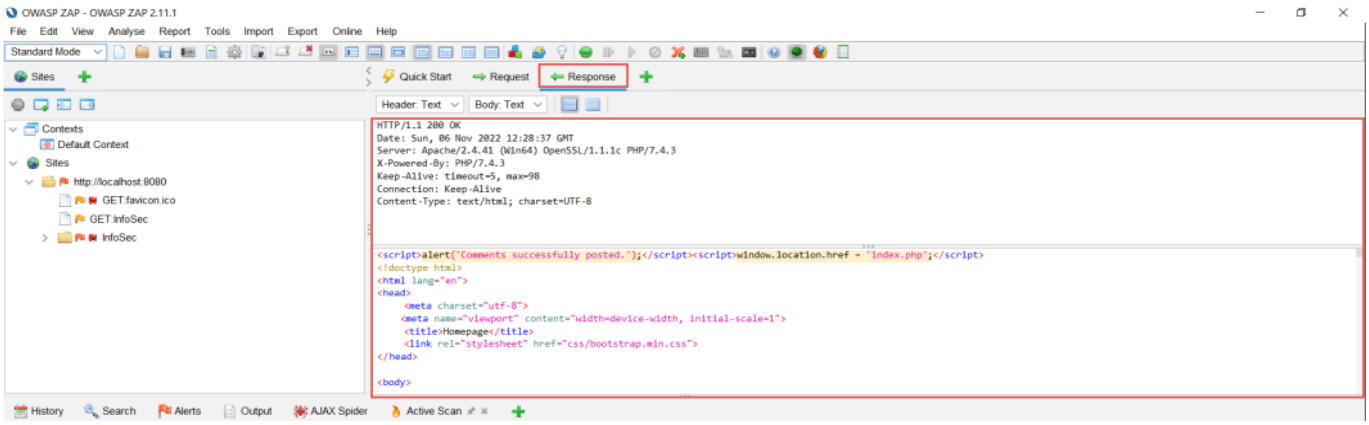
Request Tab

The Request tab shows you the data sent by your browser for the request that you have highlighted in either the Sites tab or the History tab . The top panel shows the request header, and the bottom panel shows any data also sent, for example as a result of a POST request.



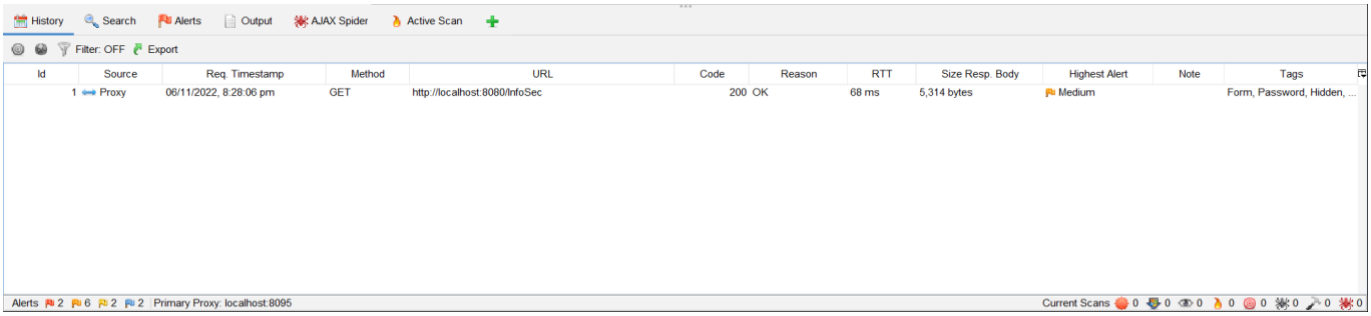
Response Tab

The Response tab shows you the data sent to your browser for the request that you have highlighted in either the Sites tab or the History tab .



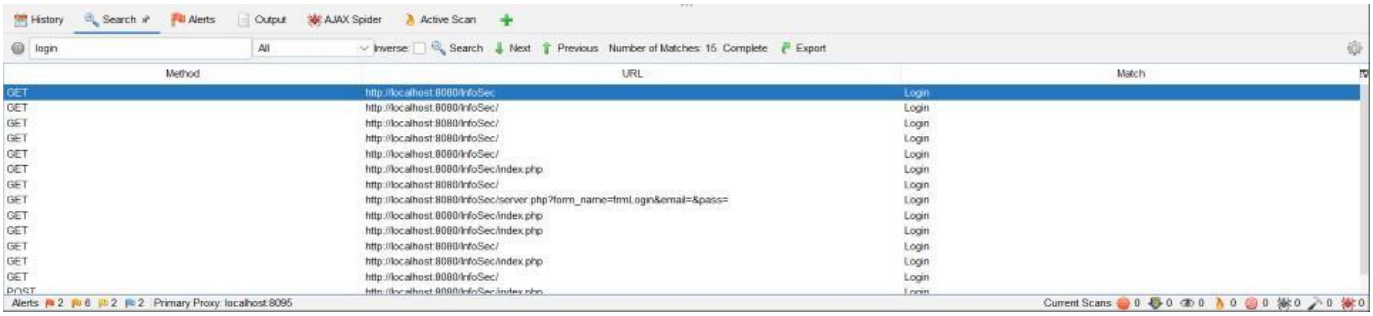
History Tab

The History tab shows a list of all of the requests in the order in which they were made.



Search Tab

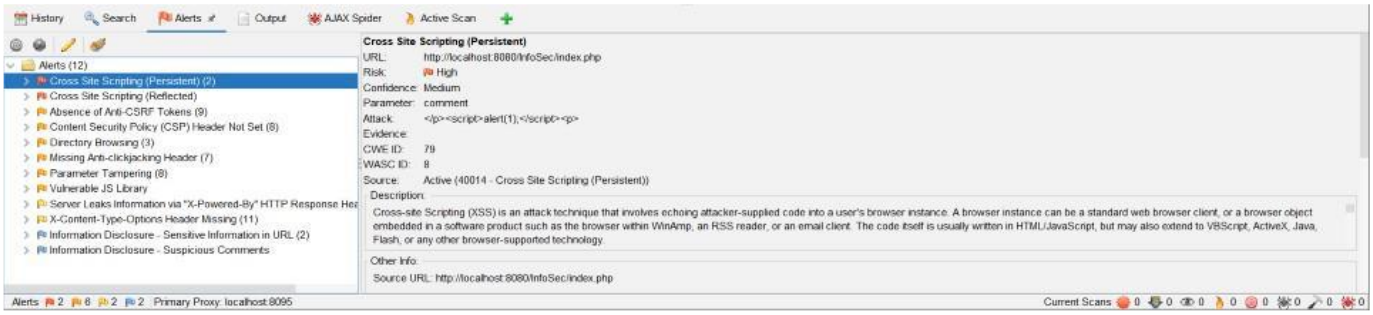
The Search tab allows you to search for regular expressions in all of the URLs, requests, responses, headers and in other functionalities provided by add-ons.



Alerts Tab

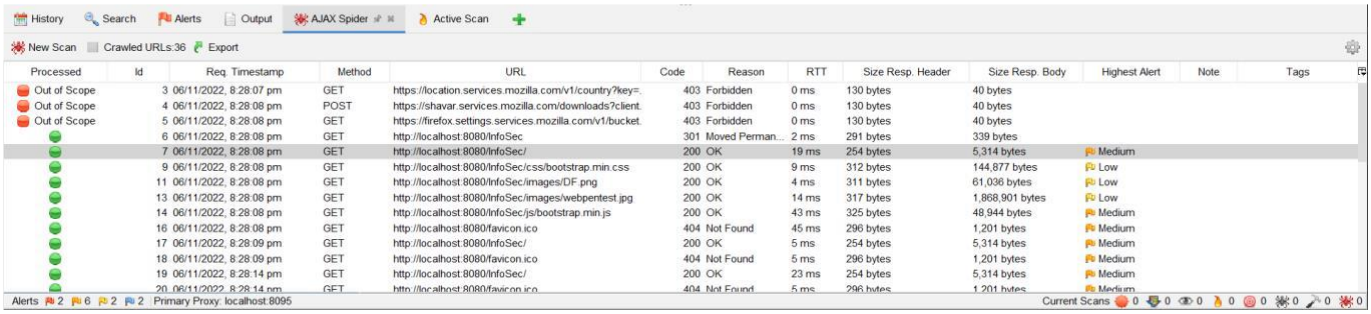
The Alerts tab show the Alerts that have been raised in this session. The alerts are displayed in a tree in risk order in the left-hand pane, and each node of the tree shows the total number of alerts underneath it.

Selecting an alert with one click will display it in the right-hand pane. Double clicking an alert will display the Add Alert dialog which will allow you to change the alert details.



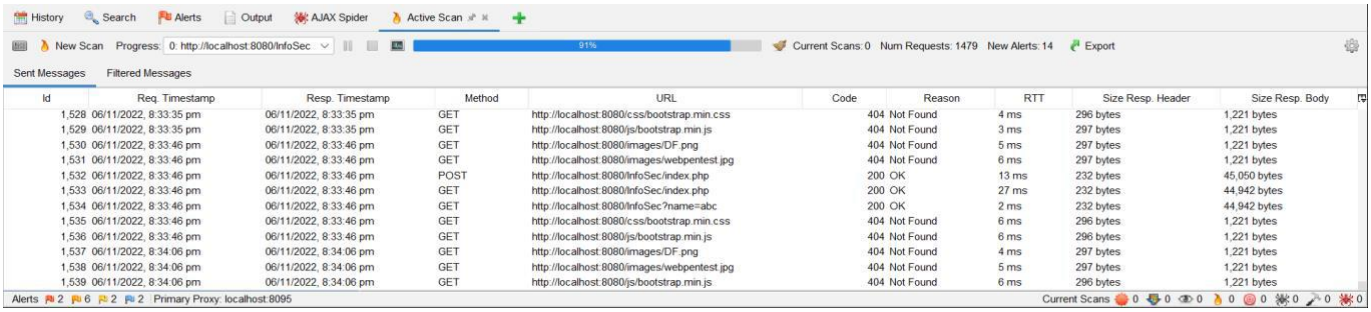
Spider Tab

The Spider tab shows you the set of unique URIs found by the Spider during the scans.



Active Scan Tab

The Active Scan tab allows you to perform an active scan.

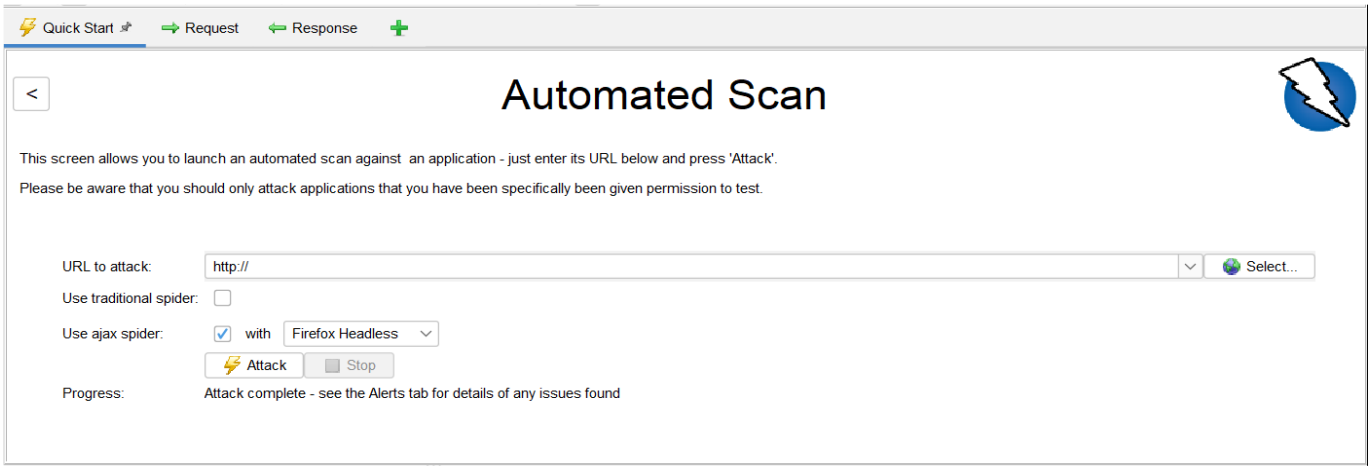


Running an Automated Scan

The easiest way to start using ZAP is via the Quick Start tab. Quick Start is a ZAP add-on that is included automatically when you installed ZAP.

To run a Quick Start Automated Scan :

1. Start ZAP and click the Quick Start tab of the Workspace Window.
2. Click the large Automated Scan button.
3. In the URL to attack text box, enter the full URL of the web application you want to attack.
4. Click the Attack button.



ZAP will proceed to crawl the web application with its spider and passively scan each page it finds. Then ZAP will use the active scanner to attack all of the discovered pages, functionality, and parameters.

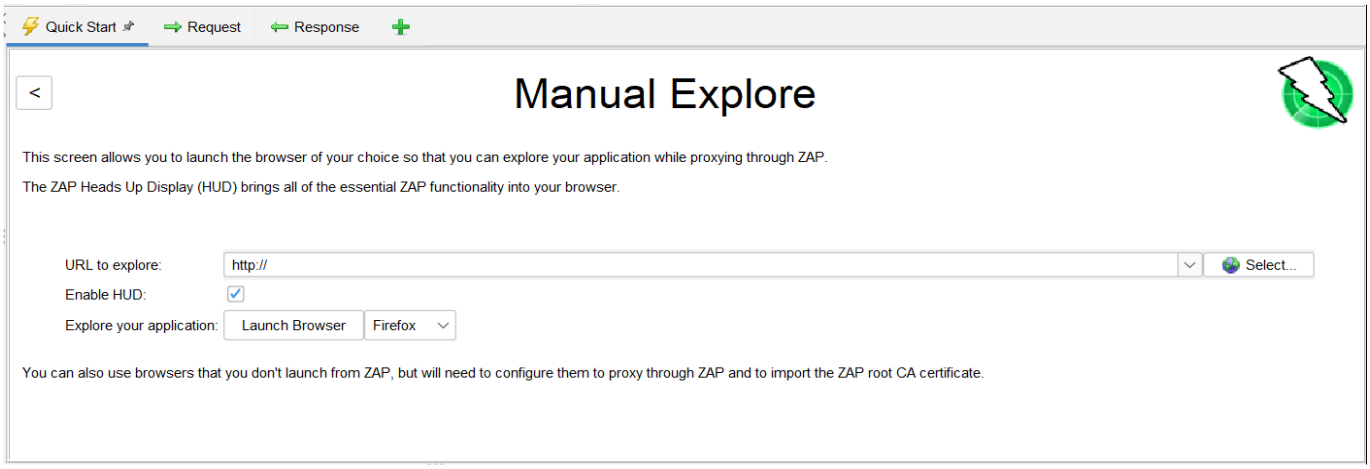
ZAP provides 2 spiders for crawling web applications; you can use either or both from this screen.

- The traditional ZAP spider which discovers links by examining the HTML in responses from the web application. This spider is fast, but it is not always effective when exploring an AJAX web application that generates links using JavaScript.
- For AJAX applications, ZAP's AJAX spider is likely to be more effective. This spider explores the web application by invoking browsers which then follow the links that have been generated. The AJAX spider is slower than the traditional spider and requires additional configuration for use in a "headless" environment.

ZAP will passively scan all the requests and responses proxied through it. Passive scanning does not change responses in any way and is considered safe. Scanning is also performed in a background thread to not slow down exploration. Passive scanning is good at finding some vulnerabilities and to get a feel for the basic security state of a web application and locate where more investigation may be warranted.

Exploring an Application Manually

The passive scanning and automated attack functionality is a great way to begin a vulnerability assessment of your web application, but it has some limitations. Spiders are a great way to explore your basic site, but they should be combined with manual exploration to be more effective. Spiders, for example, will only enter basic default data into forms in your web application but a user can enter more relevant information which can, in turn, expose more of the web application to ZAP.

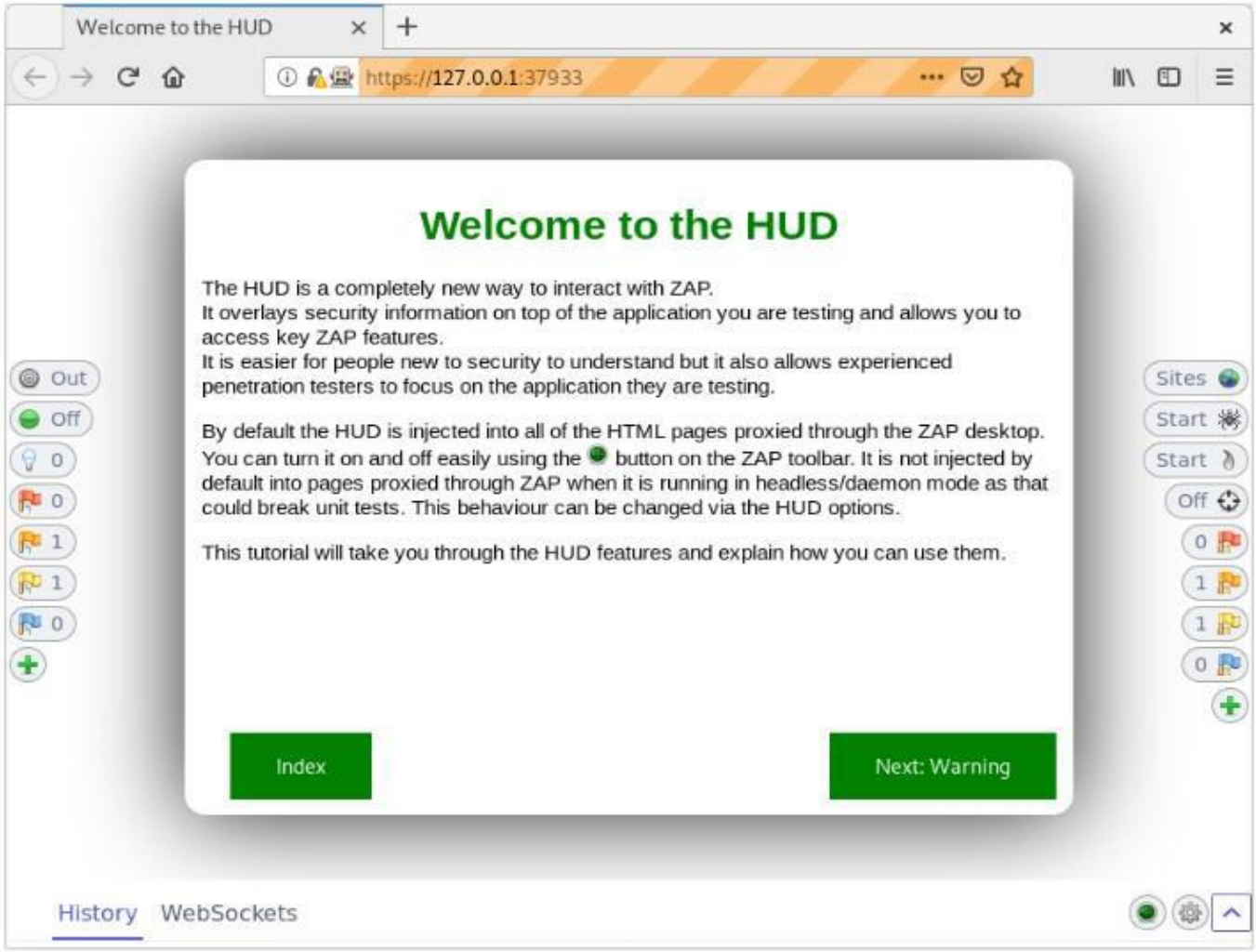


To Manually Explore your application:

1. Start ZAP and click the Quick Start tab of the Workspace Window.
2. Click the large Manual Explore button.
3. In the URL to explore text box, enter the full URL of the web application you want to explore.
4. Select the browser you would like to use
5. Click the Launch Browser button.

This option will launch any of the most common browsers that you have installed with new profiles. By default, the ZAP Heads Up Display (HUD) will be enabled. Unchecking the relevant option on this screen before launching a browser will disable the HUD.

The Heads-Up Display (HUD) is a new an innovative interface that provides access to ZAP functionality directly in the browser. It is ideal for people new to web security and also allows experienced penetration testers to focus on an applications functionality while providing key security information and functionality.



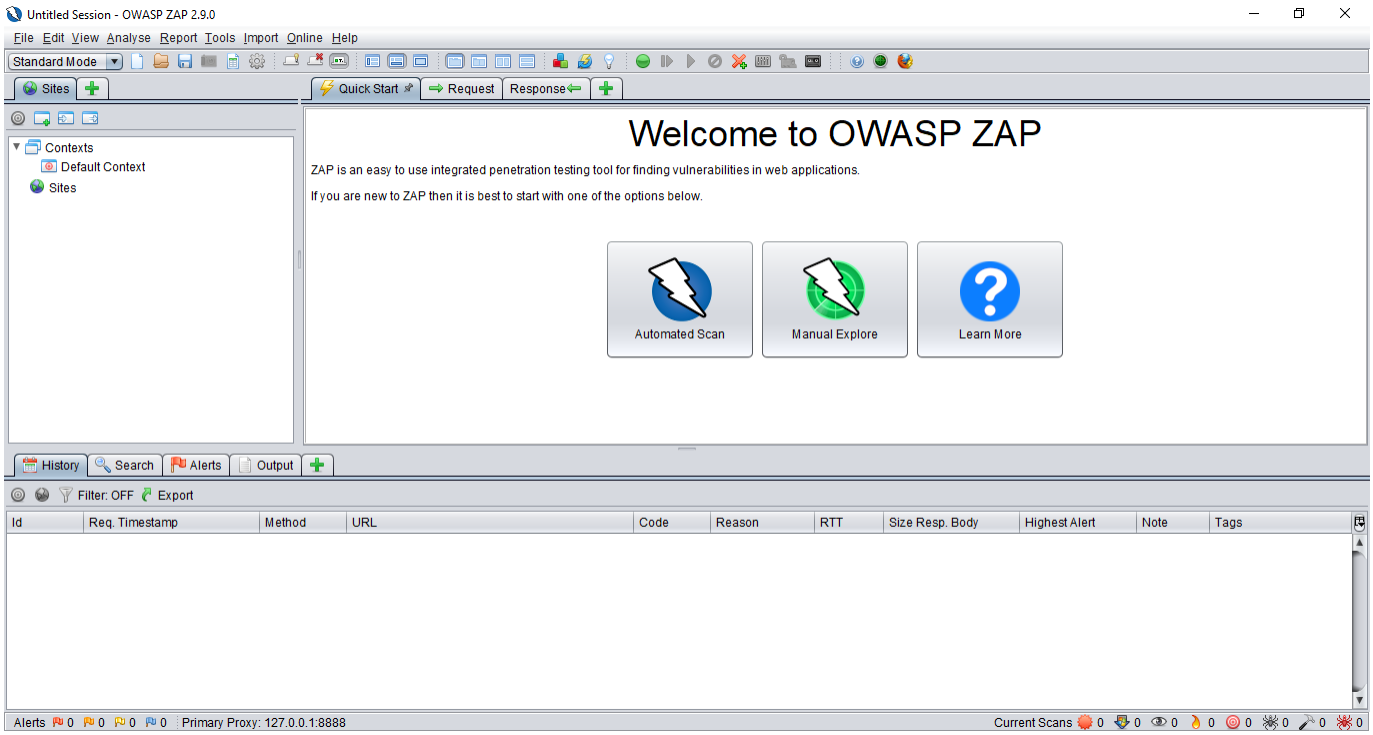
The HUD is overlayed on top of the target application in your browser when enabled via the 'Manual Explore' screen or toolbar option. Only modern browsers such as Firefox and Chrome are supported.

LABORATORY ACTIVITY
2

Vulnerability Assessment

Instructions/Directions:

Be able to conduct Automated and Manual Web Vulnerability Assessment of an Information System (refer to your Web Application Development output from the previous activity) using OWASP ZAP vulnerability scanner and list down all vulnerabilities using the provided table. You must be able to generate an HTML Report from OWASP ZAP.



Show proof of your activity completion by providing clear screenshot of your entire desktop environment based on the tasks performed. Screenshot must be included in the activity template provided in MS Teams Assignment.

Submission Note (Individual Activity)
Use the Activity template attached in MS Teams Assignment.
Use file name convention (LASTNAME_CTINASSL_SECTION_TERM_AY_Activity2.pdf).
Submit a PRINTED document and upload the Softcopy (PDF file) in MS Teams
Submit OWASP ZAP HTML Generated Report File

LESSON SUMMARY

Penetration testing replicates the actions of an external or/and internal cyber attacker/s that is intended to break the information security and hack the valuable data or disrupt the normal functioning of the organization.

Vulnerability assessment is the technique of identifying (discovery) and measuring security vulnerabilities (scanning) in a given environment.

The OWASP Top 10 is a report, or "awareness document," that outlines security concerns around web application security. It is regularly updated to ensure it constantly features the 10 most critical risks facing organizations. OWASP recommends all companies to incorporate the document's findings into their corporate processes to ensure they minimize and mitigate the latest security risks.

KEY TERMS

- Penetration Testing
- Vulnerability Assessment

REFERENCES

Online Resources

- Tutorialspoint, “Penetration Testing Vs. Vulnerability”, https://www.tutorialspoint.com/penetration_testing/penetration_testing_vulnerability_assessment.htm
- OWASP ZAP, “OWASP ZAP 2.9 Getting Started Guide”, Online Document, https://www.zaproxy.org/pdf/ZAPGettingStartedGuide-2.9.pdf
- OWASP, “OWASP TOP 10: 2025”. Website, https://owasp.org/Top10/2025/0x00_2025-Introduction/